



CHANNEL Z0 – SELF-HOSTING ON A HOMELAB (PROXMOX & TRU- ENAS)

DOC NO. RL-Z0-SELF-HOSTING-A REV. A RIPOSTE LABORATORIES

The build guide runs master control on a dedicated mini PC. This is the same station, but for people who already have a homelab humming in a closet – a Proxmox node, a TrueNAS box, a rack. Everything here is about where the playout stack lives; the tower still wants to be a VPS (that’s the whole scaling trick), and the storefront lives on Cloudflare. Only the home side changes.

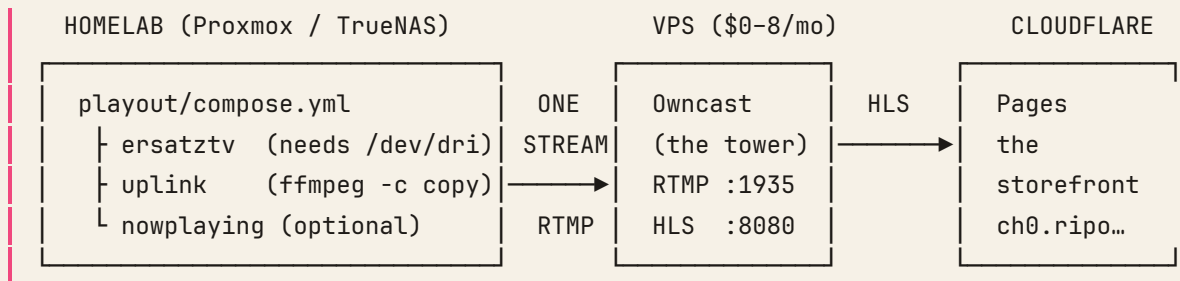
CONTENTS

▶ Where each piece runs	3
▶ Proxmox	3
Path A – LXC container (recommended)	4
Path B – VM with iGPU passthrough	4
Proxmox tips	5
▶ TrueNAS SCALE	5
1 · Datasets	5
2 · The app	6
TrueNAS tips	6
▶ Tips & tricks (any homelab)	7

The one hardware fact that decides everything below: ErsatzTV wants a **hardware H.264 encoder** (Intel Quick Sync / VAAPI, or NVIDIA NVENC). On a homelab that means handing the box's iGPU/GPU to the container. The container stack is `playout/compose.yml`; the rest of this doc is how to give it a GPU on each platform.

SEC.01

WHERE EACH PIECE RUNS



Don't self-host the tower. It's tempting to run Owncast at home behind a Cloudflare Tunnel and skip the VPS, but: RTMP ingest (:1935) can't traverse Cloudflare's HTTP proxy, and proxying a 24/7 video stream through Pages/Tunnel runs straight into Cloudflare's rules about serving lots of non-HTML traffic. The VPS is \$0 on Oracle's free tier and exists precisely so your house only ever uploads one stream. Keep the split.

SEC.02

PROXMOX

Two ways to host the playout stack. **Use the LXC path** – for Quick Sync it's dramatically simpler than VM passthrough, and it's lighter.

PATH A – LXC CONTAINER (RECOMMENDED)

An unprivileged LXC with Docker inside, and the host's `/dev/dri` bind-mounted in so the container can reach Quick Sync.

1. **Create the container.** Debian 12 template, unprivileged, give it 2 cores / 2-4 GB RAM / 20 GB root. Put the media on a separate mount (below).
2. **Enable Docker-in-LXC + GPU.** Edit `/etc/pve/lxc/<ID>.conf` on the Proxmox host and add: `# Docker needs nesting + keyctl features: nesting=1,keyctl=1 # Hand the iGPU (Quick Sync) to the container lxc.cgroup2.devices.allow: c 226:* rwm dev0: /dev/dri/renderD128,gid=44` (226 is the DRI major number; `gid=44` is usually `render` – check with `getent group render` on the host and match it. On some hosts the render group is `104` or `993`.) Restart the container: `pct restart <ID>`.
3. **Media in.** If the library lives on a NAS or another dataset, bind-mount it: `pct set <ID> -mp0 /tank/channelz0,mp=/media/channelz0`. Or keep it on the container's disk. Either way, point `Z0_MEDIA_ROOT` at it.
4. **Install Docker, then the stack.** Inside the container: `bash curl -fsSL https://get.docker.com | sh git clone <this repo> && cd channel-z0/playout cp ../.env.example .env # fill in real values docker compose up -d # ersatztv + uplink`
5. **Prove Quick Sync reached the container:** `bash docker exec -it z0-ersatztv sh -c 'ls -l /dev/dri' # renderD128 present? # optional, if vainfo is available in your image: # vainfo --display drm --device /dev/dri/renderD128` Then set the ErsatzTV FFmpeg profile to **VAAPI** and encode a channel; CPU should barely move.

PATH B – VM WITH IGPU PASSTHROUGH

Only worth it if you want full isolation or you're on NVIDIA. Intel iGPU passthrough to a VM means VFIO-binding the iGPU (and the host can't use it for console output), which is fiddly; NVENC in a VM needs the card passed through with the usual `hostpci0` + driver dance. If you're already comfortable with GPU passthrough, install Docker in the VM and run `playout/compose.yml` unchanged. If you're not, use Path A – it exists to save you this.

PROXMOX TIPS

- ▶ **Snapshot the ErsatzTV config.** Its whole brain (schedule, filler presets, library index) is the `ersatztv-config` volume. Put it on a dataset you snapshot; rebuilding the schedule by hand is the one genuinely annoying loss.
- ▶ **Pin resources.** ErsatzTV idles cheap but spikes at each program transition; 2 cores is plenty, but don't starve it to 1.
- ▶ **The uplink uses host networking** so it can reach ErsatzTV at `127.0.0.1:8409`. In an LXC that's fine. If you split them across hosts, set `Z0_CHANNEL_URL` to the ErsatzTV LAN address instead.

SEC.03

TRUENAS SCALE

TrueNAS SCALE runs Docker apps natively (Dragonfish/Electric Eel and later), and it's a natural home for the station because the **media library is already on the NAS**. Two moving parts: datasets, and a custom app.

1 • DATASETS

Create datasets (not just folders) so you get snapshots and clean ACLs:

```
tank/channelz0      # Z0_MEDIA_ROOT (the whole media tree)
tank/apps/ersatztv  # ErsatzTV config – snapshot this one
```

Build the media tree inside `tank/channelz0` with `tools/make-media-tree.sh` (run it over SSH, or recreate the folders in the UI). Give the **apps user (uid/gid 568)** read access to the media dataset and read/write to the config dataset – set the ACL in Datasets → Edit Permissions.

2 • THE APP

Install ErsatzTV + the uplink as a **Custom App** (Apps → Discover → Custom App, "Install via YAML"), pasting a trimmed `playout/compose.yml` with the dataset paths filled in. The GPU line is the important part – TrueNAS passes `/dev/dri` when you declare the device:

YAML

```
services:
  ersatztv:
    image: ghcr.io/ersatztv/ersatztv:latest-vaapi
    restart: unless-stopped
    ports: ["8409:8409"]
    volumes:
      - /mnt/tank/apps/ersatztv:/config
      - /mnt/tank/channelz0:/media:ro
    devices:
      - /dev/dri:/dev/dri
    group_add: ["44"]          # the host's 'render' gid, so VAAPI is usable
```

(Check your render gid with `getent group render` in a TrueNAS shell and match `group_add`.) Add the uplink service from the same compose file, or – if you'd rather keep the relay outside TrueNAS's app manager – run it as a tiny container via Dockge/Portainer, or on the mini PC. The now-playing bridge is optional and can live anywhere that can reach both ErsatzTV and the tower.

TRUENAS TIPS

- ▶ **Intel only, mostly.** Quick Sync via `/dev/dri` is the easy win. For NVIDIA, flip on the NVIDIA drivers in Apps settings and use `latest-nvidia`.
- ▶ **Media on spinning disks is fine** – ErsatzTV reads sequentially. Keep the **config dataset on an SSD pool** if you have one; the library index likes it.
- ▶ **Snapshot task** on `tank/apps/ersatztv`, nightly. That's your station's memory.
- ▶ **UPS:** wire the NAS's UPS into TrueNAS's built-in NUT, and the station rides through power blips like a real broadcaster (build guide, "Reliability").
- ▶ **Split option:** some people keep TrueNAS as pure storage and run playout on a separate mini PC that mounts `tank/channelz0` over NFS. Totally valid – the NAS

holds the tapes, the little box runs master control. Mount read-only.

SEC.04

TIPS & TRICKS (ANY HOMELAB)

- ▶ **Verify the encoder before you build a schedule.** A software-encoding fallback will **work** and then melt a CPU core at 1080p. Confirm VAAPI/NVENC is actually being used in ErsatzTV's transcode log on day one.
- ▶ **Updates, deliberately.** `docker compose pull && docker compose up -d` monthly-ish. Watchtower can automate it, but exclude the playout stack from auto-updates before a Ground Zero premiere – never ship on a Friday.
- ▶ **Two clocks, one truth.** Make sure the playout host's timezone is right; the sign-off ritual and the now-playing bridge both trust the wall clock.
- ▶ **Health at a glance.** Point UptimeRobot at the tower's `https://watch.<domain>/api/status`; you'll hear about an outage before a neighbour does.
- ▶ **Keep the secret in one place.** The stream key lives in `.env` (or `/etc/channel-z0.env`) and nowhere else – not in the compose file, not in a snapshot you export, not in a screenshot of your dashboard.

One stream leaves the house. The homelab just makes the house tidier.

SOURCE docs/self-hosting.tex SHA bfa7cd76299c BUILT 2026-09-07